Acoustic Tracking and Binary Detection

Project Report

Project Number: 25-1-1-3234

Students:
Mark Ruzal ID: 209959139
Roee Tabak ID: 326274651

Supervisor: Arkady Rafalovich

Project Carried Out at: Tel Aviv University Project Lab

Contents

List of Figures

List of Tables

Abstract

This project presents the design and implementation of a real-time acoustic tracking and motor-control platform that couples Direction-of-Arrival (DOA) estimation with closed-loop actuation to follow the most prominent moving sound source. A four-microphone planar array feeds a perception tier built on the Generalized Cross-Correlation with Phase Transform (GCC–PHAT): a Time Difference of Arrival (TDOA) is estimated for each of the six microphone pairs and fused by a confidence-weighted least-squares solver into an azimuth–elevation bearing, which is then smoothed using exponential moving average (EMA) to suppress jitter. In parallel, a dynamically thresholded Short-Time Fourier Transform performs binary, presence/absence detection of narrowband acoustic signals by comparing each spectral peak against the local statistics of its surrounding bins and confirming it through temporal persistence. The dominant persistant frequency is used to determine the center of the BPF filter, prior to the DOA estimation part. The resulting coordinates are streamed over a serial link to an STM32 microcontroller, whose hardware tier ingests the data through a double-buffered (ping-pong) Interrupt Service Routine that is race-free and self-recovering. A velocity-mode PID controller with anti-windup and shortest-path angle wrapping drives an L6474 stepper pair across pan and tilt axes. Experimental results demonstrate a Root Mean Square Error (RMSE) of 1.84° in azimuth (ϕ) and 1.93° in elevation (θ) — within the predefined 5° tolerance — together with over 6 dB of noise rejection in the perception front-end. The closed-loop platform reacts to a change in source position in under one second and holds sub-degree steady-state pointing error in real time.

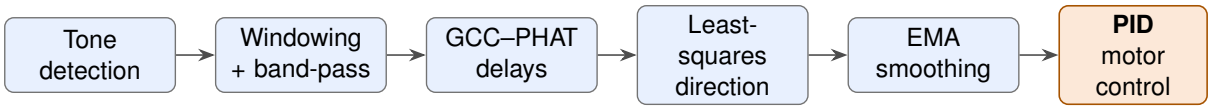

Figure 1: High-level signal flow of the platform: acoustic perception (tone detection through EMA smoothing) feeding the PID motor-control stage.

1 Introduction

1.1 Goals of the project

- Develop a robust Direction-of-Arrival (DOA) algorithm able to track drone-like acoustic signatures even in noise, with the target signal only about 6 dB above the surrounding noise floor.
- Implement a binary detection system that reliably recognizes and logs specific generated tones (400, 800, 1200, and 1600 Hz), correctly identifying their presence more than 90% of the time through temporal persistence.
- Achieve angular tracking with an error of less than 5°.
- Develop control firmware for a motorized pan–tilt platform, together with a supervising GUI, that orients itself toward the acoustic source with a reaction time of less than one second.

1.2 Motivation

Acoustic tracking is highly susceptible to real-world disturbances, and a system that loses its target the moment background noise rises is of little practical use. Developing an algorithm that remains robust against environmental noise is therefore the primary driver for this work, ensuring reliable operation in both simulated and real-life scenarios. A platform that can passively localize and follow a weak, moving acoustic source — such as a small drone — using only a compact microphone array is valuable precisely because it must succeed under exactly the low signal-to-noise conditions where naive methods fail.

The PoC model of an acoustic sound source tracking solution, this works aims to provide, is a lightweight and cheap alternative to expensive radar object detection systems. Moreover, relying on sound may, in some cases, allow for direction sensing even under obstructed line of sight.

1.3 The approach to solving the problem

The system is divided into two cooperating tiers: an audio signal-processing tier that estimates the source direction, and a hardware control tier that physically steers toward it. To validate the DOA algorithm before deployment, incoming plane waves are simulated with additive Gaussian white noise, allowing the angular accuracy to be measured against a known ground truth across controlled signal-to-noise ratios. Binary detection is achieved by monitoring the short-time spectrum and confirming a tone only once it persists above a locally adaptive threshold, which lets the system identify — and log — the precise moments at which each frequency appears and vanishes. The estimated bearing is then streamed to the control tier, which drives the platform toward the source in real time.

1.4 Comparison against existing work

The algorithms underlying this work — GCC–PHAT delay estimation, adaptive detection, and PID control — are well established, and the project builds on existing papers rather than improving any single algorithm; the contribution lies in their integration and operating point. Where most acoustic DOA studies are estimation-only, reporting accuracy on a static array, our system closes the loop through a physical actuator in real time — and because the array rides the pan axis, each azimuth is measured in a moving frame, which we resolve with null-seeking control (i.e., the x axis in our coordinate system, as described in the theoretical background, points at the azimuth direction). Rather than localizing at high SNR where most methods succeed, the front-end is engineered for weak sources roughly 6 dB above the noise, making robustness the design center through a tone-locked band-pass, a locally adaptive detector, and temporal persistence. Detection and localization are also fused rather than assumed: the persistence detector first decides whether, and at which frequency, a tone is present, and that decision drives the band-pass and delay estimation that follow. Finally, the whole chain runs on commodity hardware — a four-microphone array and an STM32 stepper platform — yet holds sub-degree pointing error at a sub-second reaction time, well below the cost of typical systems.

2 Theoretical background

2.1 Time Difference of Arrival (TDOA) and GCC-PHAT

The core of the Direction of Arrival (DOA) engine relies on calculating the time delay between microphones. Let the signal received at a reference microphone be $x_1(t) = S(t) + n_1(t)$ and the signal at a second microphone be $x_2(t) = S(t - \tau_d) + n_2(t)$, where τ_d is the delay we want to find and $n_1(t), n_2(t)$ represent the noise, reverberations and other unwanted disturbances detected by microphones 1 and 2, respectively.

Regularly, one might compute the standard cross-correlation:

$$R_{12}(\tau) = \int_{-\infty}^{\infty} x_1(t)x_2(t-\tau)dt \tag{1}$$

However, this time-domain approach is highly sensitive to echoes, reverberations, and noise. In a real room, sound bounces off walls; these reflections create peaks and valleys in the frequency spectrum, which translate to multiple false peaks in the time domain, making it difficult to find the true τ_d.

To resolve this, the system employs the Generalized Cross-Correlation Phase Transform (GCC-PHAT) [1, 5, 6]. The GCC framework applies a weighting function $W(f)$ to the cross-power spectral density $G_{12}(f) = X_1(f)X_2^*(f)$ to emphasize reliable frequencies and suppress corrupted ones. By choosing the PHAT weighting $W_{PHAT}(f) = \frac{1}{|G_{12}(f)|}$, the formula becomes:

$$R_{GCC-PHAT}(\tau) = \int_{-\infty}^{\infty} \frac{G_{12}(f)}{|G_{12}(f)|} e^{j2\pi f\tau} df \tag{2}$$

Because $\frac{G_{12}(f)}{|G_{12}(f)|} = e^{j\angle G_{12}(f)}$, the magnitude of every frequency is normalized to unity. In an ideal noiseless scenario, where $x_2(t)$ is just a shifted version of $x_1(t)$, the integral

evaluates to $\delta(\tau - \tau_d)$. This produces a distinct, sharp spike *only* at the correct delay when the delayed signals match.

Since the microphones are positioned at coordinates $\mathbf{r}_m = \frac{1}{\sqrt{2}}[\pm D/2, \pm D/2, 0]^T$, the maximum possible delay is $\sqrt{2}\frac{D}{c}$ between diagonal pairs, and $\frac{D}{c}$ between adjacent pairs. Hence, the estimated delay $\tau_d = \text{argmax}\{R_{GCC-PHAT}(\tau)\}$ is searched within $[-\tau_{max_{ij}}, \tau_{max_{ij}}]$, where $\tau_{max_{ij}}$ is the maximum delay of the pair ij.

To surpass the discrete sampling limits of the Fast Fourier Transform (FFT), the code applies parabolic interpolation around the discrete peak index (idx) using its neighboring correlation values to find a continuous sub-sample delay offset δ:

$$\delta = \frac{1}{2}\frac{R[idx-1] - R[idx+1]}{R[idx-1] - 2R[idx] + R[idx+1]} \tag{3}$$

2.2 Weighted Least Squares DOA Estimation

A robust Weighted Least Squares (WLS) solution to account for the square geometry of the 4-microphone planar array [6].

For any microphone pair (i, j), the geometric vector between them is $\mathbf{A}_k = \mathbf{r}_j - \mathbf{r}_i$. The theoretical relationship between the measured delay vector $\boldsymbol{\tau}$ (whose entries are the delays between each microphone pair) and projection of the unit vector $\mathbf{u}$, pointing towards the sound source, onto the xy-plane $\mathbf{u}_{xy} = (\sin\theta\cos\phi, sin\theta\sin\phi)$ is:

$$\mathbf{A}\mathbf{u}_{xy} = c\boldsymbol{\tau} \tag{4}$$

Where c is the speed of sound and $\mathbf{A}$ is a 6×2 matrix representing all 6 pairwise combinations of the 4 microphones ($\binom{4}{2}$). Because this is an overdetermined system (6 equations for 2 variables), the system is solved by the Least Squares solution:

$$\mathbf{u}_{xy} = (\mathbf{A}^T\mathbf{W}\mathbf{A})^{-1}\mathbf{A}^T\mathbf{W}\mathbf{d} \tag{5}$$

Here, $\mathbf{d} = c\boldsymbol{\tau}$, and $\mathbf{W}$ is a diagonal weighting matrix. To penalize noisy measurements, the code dynamically calculates the weights based on peak clarity: $W_{kk} = 1 - \left|\frac{c_2}{c_1}\right|$, where c_1 is the primary correlation peak and c_2 is the secondary peak. A strong secondary peak indicates high reverberation, thus lowering the weight of that specific microphone pair.

Finally, the 3D spherical angles are extracted from the xy-plane projection:

$$\phi = \text{atan2}(u_y, u_x) \tag{6}$$

$$\theta = \arcsin(\|\mathbf{u}_{xy}\|) \tag{7}$$

2.3 Dynamic Statistical Thresholding for Binary Detection

Rather than using static amplitude thresholds, the binary detection script evaluates the Short-Time Fourier Transform (STFT) dynamically (or simply an FFT). The algorithm begins by capturing all the local maxima of the FFT. Then, for a each maxima frequency bin f_{peak}, a local spectral neighborhood is defined. The local mean (μ) and standard deviation (σ) are computed.

A tone is only considered a valid candidate if its magnitude $|FFT(f_{peak})|[dB]$ satisfies:

$$|FFT(f_{peak})|[dB] > \mu + 1.3\sigma \tag{8}$$

Furthermore, a temporal state-machine enforces a "persistence constraint." The signal must consecutively satisfy this statistical bound for a duration of T_{frames} (2.0 seconds) to be validated as a binary signal, effectively filtering out transient noise spikes. To account for minor frequency fluctuations (such as Doppler shifts) around the initially determined f_{peak}, the condition in Eq. (8) was enforced within a narrow frequency window (30 Hz) in the subsequent time frames.

2.4 Motor Control

After the DOA algorithm calculates the estimates ϕ_s, θ_s, they are transmitted to the motor control firmware algorithm, which implements a PID control on each of the tilt and pan axes separately. The pan angle of the rotating platform is denoted by α, and the tilt angle, by β.

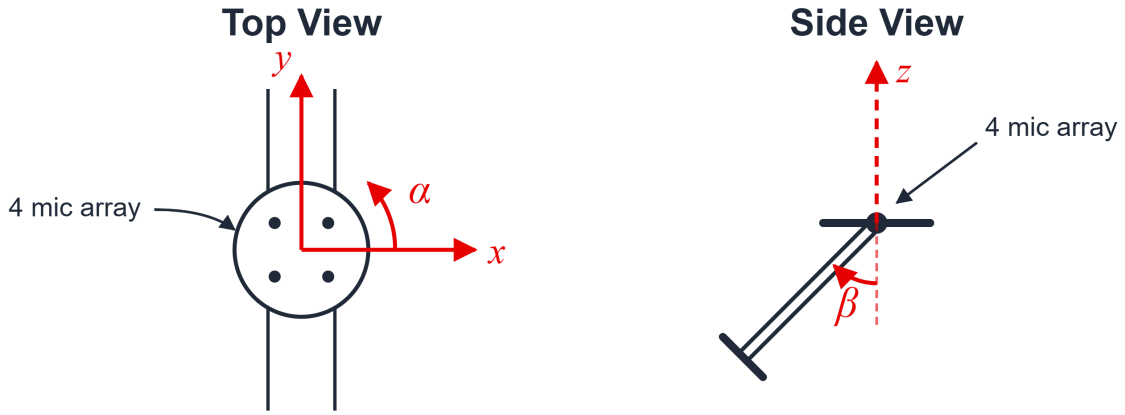

Figure 2: The platform setup illustration.

The goal is to orient the platform, with the microphone array attached to the pan axis, in such a way that the center of the platform will be directly in front of the sound source, i.e., on the straight line connecting the intersection point of the pan and tilt rotation axes with the center of the sound source. Therefore, supposing the platform is currently aligned to some (α, β), the target pan and tilt angles must be:

$$\alpha_t = \alpha + \phi_s, \;\; \beta_t = \theta_s \tag{9}$$

Where in the first equation we used the coordinate transformation between the $x_s y_s$ local plane of the microphone array, which rotates along with the pan axis, and the stationary xy plane in which α and β are defined (see the figure below).

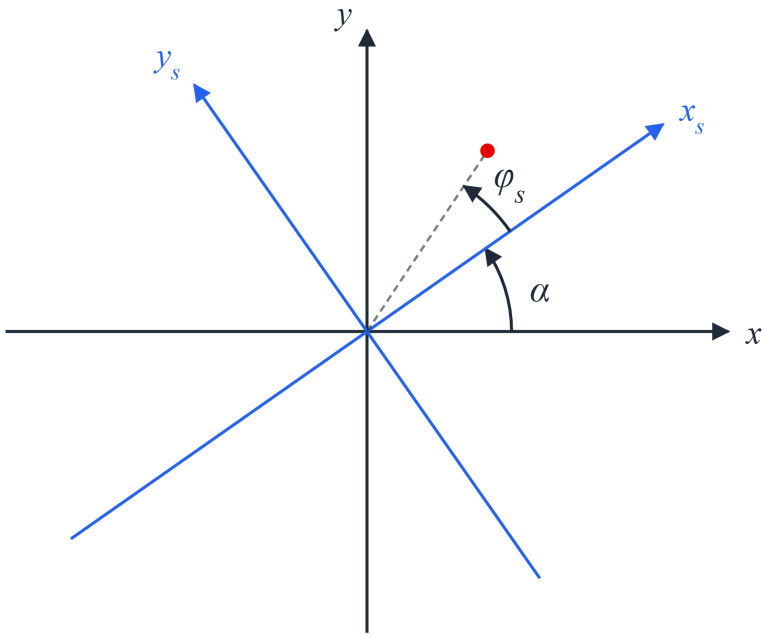

Figure 3: The relationship between the two coordinate systems — the stationary axes which are used to determine the track the motor positions, and the local microphone array axes that rotate with the pan and relative to which the estimated ϕ_s, θ_s are defined. $z_s = z$.

The PID algorithm that brings the platform to the correct position is depicted below (all variables are in continuous time for simplicity):

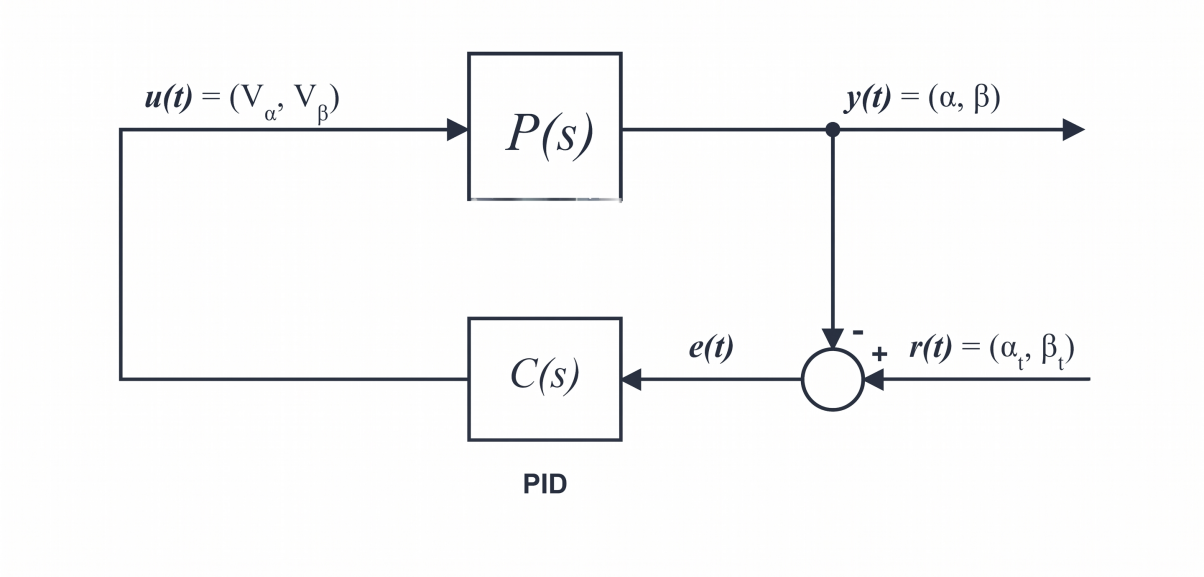

Figure 4: v_α, v_β denote the velocity input commands to the two stepper motors controlling the pan and tilt axes. $P(s)$ is the plant — stepper motors controlled by the velocity commands through the drivers. $C(s) = K_p + K_i \cdot \frac{1}{s} + K_d \cdot s$.

3 Simulation

In this chapter the simulation environment will be described. Its purpose is to validate the Direction-of-Arrival (DOA) pipeline against a known ground truth without physical hardware: the simulator synthesises the exact multi-microphone signals that a source at a chosen direction would produce, corrupts them with noise, and feeds them to the same estimation code used for live audio. Because the true direction is known by construction, the estimation error can be measured directly.

The simulator is not a separate program but a mode of the live Python system — the user chooses how the sound data is acquired — read from the ReSpeaker array in live mode or generated mathematically in simulation. Everything downstream — TDOA estimation, the DOA solve, and smoothing — is identical in both modes, so the simulation exercises the real algorithm rather than a stand-in, and any figure it produces reflects the deployed code path.

Each block is built from one broadband base signal — white Gaussian noise, chosen because it gives the delay estimator usable phase across the whole band — which plays the role of the acoustic source. For a chosen direction (ϕ, θ), with ϕ the azimuth and θ the polar angle measured from the z-axis (the array's normal), the propagation unit vector is

$$\mathbf{u} = \big(\sin\theta\cos\phi,\ \sin\theta\sin\phi,\ \cos\theta\big),$$

and a microphone at known position $\mathbf{r}_m$ on the circular array sees the plane wave with geometric delay $\tau_m = -\,\mathbf{r}_m \cdot \mathbf{u}/c$. Rather than rounding to whole samples, this delay is applied in the frequency domain via the Fourier shift theorem — the base spectrum is multiplied by $e^{-j2\pi f \tau_m}$ before being transformed back — so every channel carries the same waveform displaced by its true sub-sample delay. Independent Gaussian noise is then added to each channel. This is what makes the test meaningful: the source is perfectly correlated across microphones (differing only by delay) while the noise is uncorrelated, so the estimator must recover the correlated delay from beneath uncorrelated noise. The noise amplitude is tunable, so the SNR can be swept to characterise robustness.

The true direction is driven along one of several parametric trajectories so that each angular degree of freedom can be exercised in isolation or together:

- **Circle** — azimuth ϕ sweeps a full 360° at constant elevation, isolating azimuth tracking.
- **Arc** — azimuth is fixed while elevation oscillates between horizon and zenith, isolating elevation tracking.
- **Spiral** — both angles vary at once, exercising the coupled two-dimensional case.

Furthermore, the underlying trajectory generation code is completely general and can be easily adjusted to simulate any custom trajectory we want.

When a trajectory completes, the environment computes summary statistics (mean, MAE, RMSE, standard deviation, and maximum error) separately for azimuth and elevation. Because the ground truth is exact, these figures are an unbiased measure of the algorithm's angular accuracy.

4 Implementation

In this chapter the implementation is described, along with the reasons behind the main design choices. The system is built from two parts that run on separate hardware and talk to each other over a serial cable. The first part runs in Python on a regular computer and works out the direction the sound is coming from. The second part runs in C on an STM32 microcontroller and turns the pan–tilt platform to face that direction. Figure 5 shows how data moves through the system.

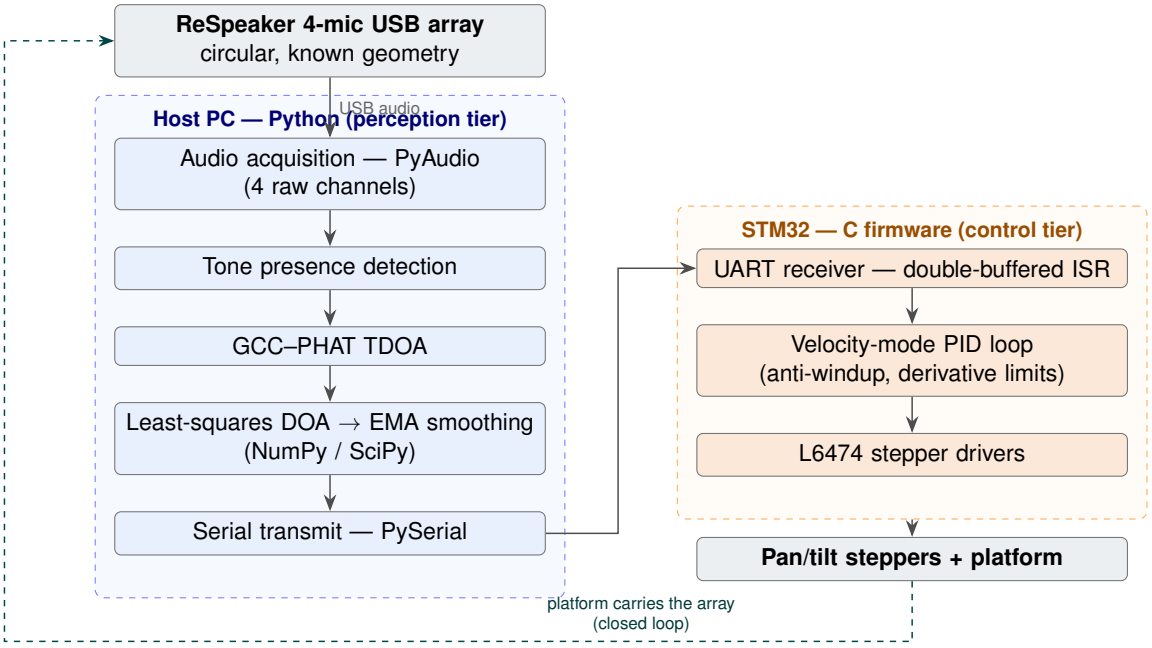

Figure 5: The system: a two-tier implementation. A Python perception tier on a host PC estimates the source bearing and streams it over UART to an STM32 control tier, which drives the pan/tilt steppers. Because the array is mounted on the platform, the arrangement forms a closed loop.

Splitting the work across two devices is a deliberate choice. The computer handles the heavy audio mathematics — the Fourier transforms, the GCC–PHAT delay calculation, and the direction estimate — and Python is well suited to this, with ready-made libraries for the signal processing, easy access to the USB microphone, and convenient tools for plotting and running the offline simulation. The microcontroller, on the other hand, is responsible for moving the motors smoothly and on time. Keeping the two separate also means that a delay or hiccup on the computer cannot disturb the motor movement: the microcontroller simply keeps aiming at the last direction it was sent, and if the connection drops entirely, a timeout stops the platform so it does not run away. Finally, because the microphone array is mounted on the moving platform, turning the motors changes what the array hears — the system feeds back on itself — and the control code handles this by always turning the shorter way around and driving the measured direction toward zero.

Hardware Description

The hardware comprises four elements: the microphone array, the host computer, the microcontroller with its stepper-driver expansion, and the motorised pan–tilt platform.

The acoustic front end is a ReSpeaker USB microphone array carrying four MEMS microphones in a circular arrangement of known radius. It streams the four raw microphone channels to the host over USB at a fixed sample rate. Its fixed, known geometry is precisely what makes the geometric direction solve possible, since the estimator relies on the inter-microphone spacings.

The host computer runs the perception software and exposes two interfaces: a USB audio input from the array and a serial (UART) output to the microcontroller.

The control hardware is an STM32 Nucleo development board with an STM32F401RE microcontroller, equipped with X-NUCLEO-IHM01A1 stepper-driver expansions based on the L6474 driver [3, 4]. Two drivers are used — one for the pan axis and one for the tilt axis — addressed over SPI by the board's motor-control library. The two axes use different gear ratios (a full revolution corresponds to 3345 microsteps on pan and 16576 on tilt), which the firmware accounts for in its step-to-angle conversion.

The mechanical platform is a two-axis pan–tilt rig: the pan axis rotates the whole assembly in azimuth, carrying the microphone array with it, while the tilt axis sets the elevation. Mounting the array on the moving platform is what closes the perception–action loop. The microcontroller communicates with the host over its UART at 115200 baud, receiving the estimated bearing as short ASCII messages.

Software Description

The **perception tier** is implemented in Python 3 using NumPy and SciPy for the numerical signal processing, PyAudio for multichannel capture from the array, PySerial for the serial link, and Matplotlib for live visualization and end-of-run statistics. In order to implement the digital Butterworth Band Pass Filter the scipy.signal butter function was used. The perception stage is organized into a small set of modules with clearly separated responsibilities.

The signal-processing module contains the core algorithms — the per-pair GCC–PHAT time-delay estimation with sub-sample peak refinement, the weighted least-squares direction-of-arrival solve, and the temporal (EMA) [8] smoothing of the resulting bearing — together with the binary tone-detection routine. The latter compares each spectral peak against the local statistics of its neighborhood and tracks it with a hysteresis counter: the counter is incremented on every frame in which a candidate clears the adaptive threshold and decremented when it does not, so a tone is confirmed only once the count reaches a set level and is released at a lower one. This two-level (set/reset) behavior is what gives the detector its noise immunity — rejecting transient spikes while tolerating brief dropouts of a genuine source. The constants module contains the fixed parameters — array geometry, sample rate, block size, detection thresholds, and the smoothing factor — so that tuning is confined to one place, and the interface module formats the estimated angles into the serial protocol and transmits them to the microcontroller.

The **control tier** algorithms are implemented in C using the STM32CubeIDE (Integrated Development Environment), together with a motor-control library for the L6474 drivers [3, 4].

The control flow starts from the moment the perception tier finished an angle estimation — The Python code establishes a serial UART connection with STM32 and sends newline-terminated ASCII line of the form "P:<phi>,T:<theta>", with the azimuth and elevation in radians, at 115200 baud.

The STM32 receiving protocol is based on a double-bufferered (ping-pong) ISR (Interrupt Service Routine), so reception never races with processing and malformed or oversized messages are discarded and resynchronised.

The parsed incoming string message is then used to update the α_t, β_t, according to Eq. 9. The current motor pan and tilt positions in steps are obtained using the BSP_MotorControl_GetPosition() command and converted to radians accounting for the corresponding gear ratio and the 1/16 microstepping setting.

Subsequently, the velocity PID controller starts minimizing the errors $e_{pan} = \alpha_t - \alpha$, $e_{tilt} = \beta_t - \beta$ (see Fig. 4), and moving the motors using the BSP_MotorControl_Run() command. A data watchdog keeps track of the time elapsed from the latest message acquisition and stops the motors if no valid message arrives within a fixed interval. Also, to prevent motor position unwinding, both the current angle α, which is based on the raw step count, and the error, undergo a modulo 2π conversion to the $(-\pi, \pi]$ range.

In order to mitigate overshoot due to wind-up, a clamp was set on the cumulative error, which starts incrementing from the start of the run. Moreover, this error is reset to zero upon receiving a command to move to an angle which is more than 5° away from the current position. The derivative error ($\frac{de}{dt}$) was also limited in code to prevent jerky or unsafe behavior.

A balance between reaction time, overshoot and steady state error minimization was achieved by manually tweaking the PID parameters, as well the motor minimal and maximal speeds, acceleration and deceleration rates.

5 Analysis of results

This chapter compares the simulation and real-time performance of the system across its two functions: direction-of-arrival tracking and binary tone detection.

5.1 Direction-of-arrival tracking

The tracking accuracy was measured in simulation, where the true source direction is known exactly and the estimation error can be computed directly. Across the test trajectories the system achieved the errors summarised in Table 1, both comfortably within the predefined 5° tolerance.

Table 1: DOA tracking error.

Quantity	**Simulation**	**Real time**
Azimuth RMSE (ϕ)	1.84°	$< 5^\circ$ (visual)
Elevation RMSE (θ)	1.93°	$< 5^\circ$ (visual)
Tolerance	5°	5°

Figure 6 shows a representative run: the azimuth tracking error over a full circular trajectory. The error stays bounded throughout, with an RMSE of $1.84°$, a mean absolute error of $1.76°$, and a worst-case error of $2.70°$ — well inside the tolerance.

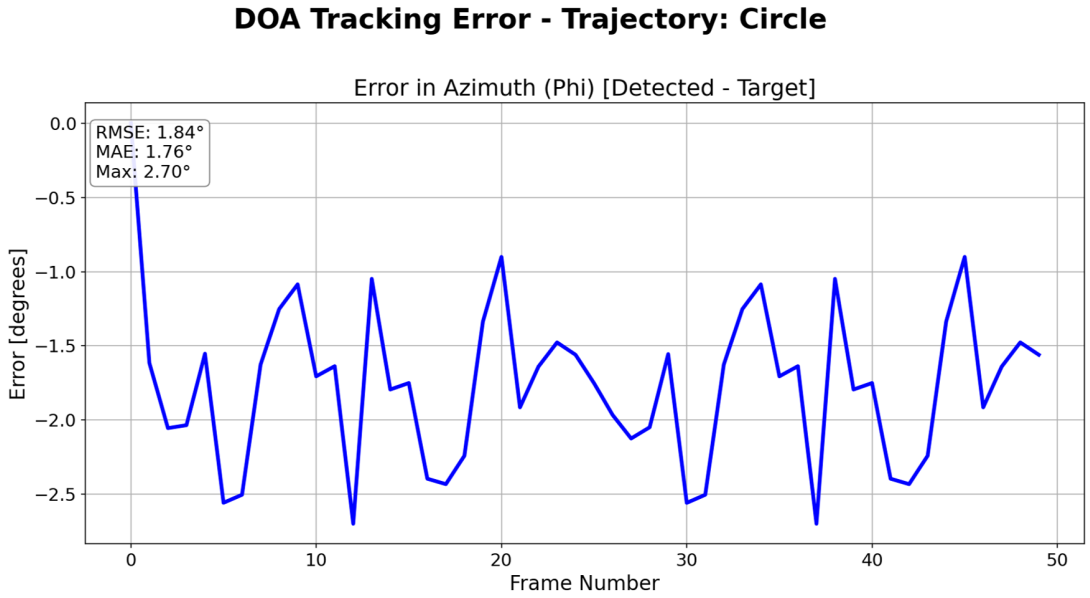

Figure 6: DOA tracking error over a circular trajectory (azimuth). The error remains bounded across the full sweep, with RMSE $1.84°$, MAE $1.76°$, and a maximum of $2.70°$.

In real-time operation the true direction is not known precisely, so the error could only be assessed visually (we mounted a camera on the platform and made sure the sound source is located in the center of the camera feed) rather than measured numerically. Under these conditions the platform consistently pointed to within approximately $5°$ of the source on both axes, in agreement with the simulation figures. The platform reacted to a change in source position in under one second.

5.2 Binary tone detection

The binary detector was evaluated by playing the four target tones and observing the real-time spectrogram, shown in Fig. 7. Each tone is correctly tracked while present (green markers) and released shortly after it stops; the labeled lag is the delay between a tone ceasing and the algorithm declaring it gone.

All four tones were detected and tracked. The dropped-signal lags are summarized in Table 2. The detector locks onto the measured spectral peak rather than an assumed frequency, so the tracked values (391, 797, 1203, 1594 Hz) differ slightly from the nominal generated tones - this is expected. All lags are well under one second.

The lag is governed by the persistence (hysteresis) counter, which must count down before a tone is released; this is the deliberate cost of the noise immunity it provides, and the sub-second values confirm the trade-off is well balanced. The analysis of false positives was outside the scope of this project.

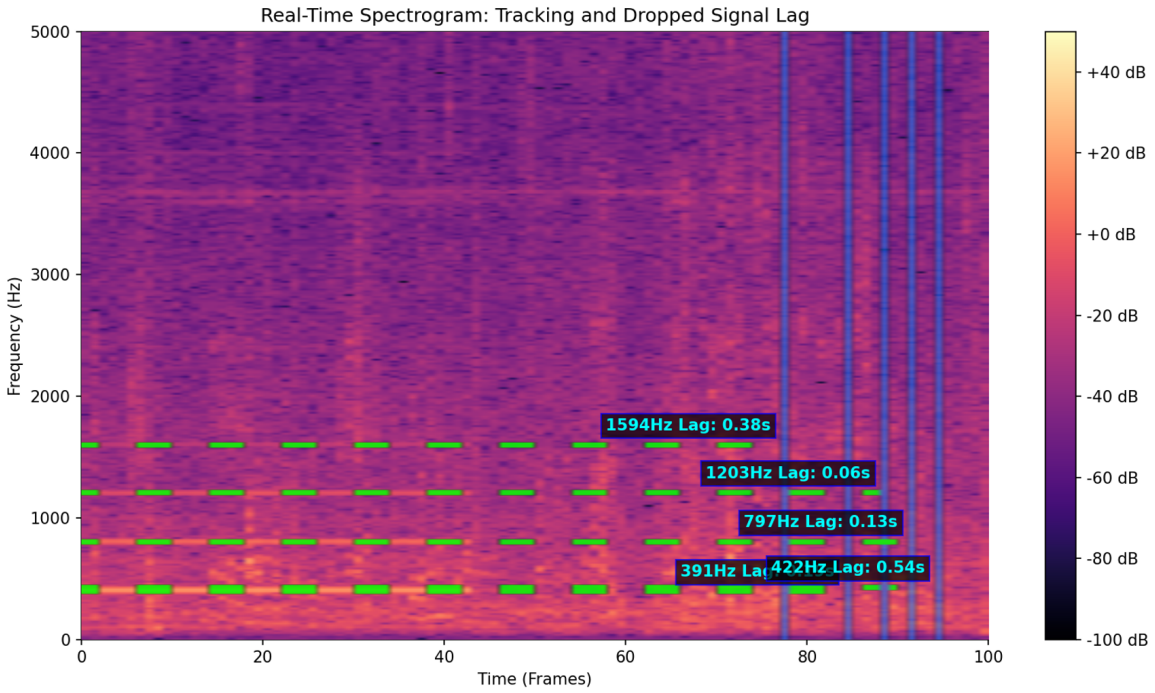

Figure 7: Real-time spectrogram of the binary detector. Green markers indicate a tone currently flagged as present; the labelled lag is the delay from a tone stopping to its removal.

Table 2: Dropped-signal detection lag per tone.

Nominal tone [Hz]	Detected [Hz]	Lag [s]
400	391	0.54
800	797	0.13
1200	1203	0.06
1600	1594	0.38

6 Conclusions and further work

6.1 Conclusions

1. The DOA algorithm tracked a drone-like acoustic source accurately in simulation, achieving azimuth and elevation RMSE of 1.84° and 1.93° — both within the 5° target — and was visually confirmed to remain within 5° in real-time operation.

2. The binary detector successfully recognised all four generated tones and reported their disappearance with a lag well under one second on every tone.

3. The motorised pan–tilt platform, driven through the velocity-mode PID controller and supervising GUI, oriented toward the source with a reaction time under one second.

4. Taken together, the system meets its goals: closed-loop acoustic tracking and tone detection at low SNR, on commodity hardware, in real time.

6.2 Further work

Several factors can improve the accuracy of the detection, tracking, and performance of the control of the motors:

The first is having a longer time window, since we will receive more data for the correlation integral, and by the other side of the same coin, we will have a better resolution in the frequency domain. There is a tradeoff between the length of the time window and the reaction time, as longer T means longer recording time, and thus slower reaction time.

The second is the sampling rate, F_s. Because the raw delay resolution is one sample time interval, $1/F_s$, a higher sampling rate lets the estimator distinguish smaller differences in arrival time, which directly improves the temporal — and hence angular — resolution. Although, supposing F_s increases while T is kept constant, the block size of the FFT grows accordingly and this translates into longer computation time of the math, as it depends on solely on the length of the Numpy arrays the code processes.

The third factor, and the one that motivates a new array, is the physical spacing between the microphones, D. A wider array spreads the same range of arrival angles over a wider range of measurable delays: a given angular movement of the source produces a delay change spanning many more sampling intervals. With the present compact array the spacing is so small that even a large angular displacement of the source shifts the delay by only a handful samples; a wider array would turn the same motion into a shift of many samples, and thus improving angular resolution. Conversely, if the spacing grows too large, the maximum pairwise delay $\sim D/c$ may exceed $\frac{\mathrm{T}}{2}$ and due to the circular wrap-around of the DFT, the estimate will be off by T. While this specific problem can be fixed by zero-padding prior to the FFT, if the delay grows too large, the two microphone channels will start losing overlap significantly, and this will compromise the delay estimation up to the point where the delay reaches T, and the overlap vanishes completely. In addition, the GCC–PHAT estimate is obtained through a DFT-based (circular) cross-correlation whose validity rests on the periodicity assumption of the transform. Large delays relative to the window violate this assumption may corrupt the estimate. A better array would therefore enlarge the spacing while respecting this upper bound, and could add microphones as well: more microphones give more independent pairwise measurements, averaging down noise and improving robustness rather than raw resolution.

Additional points for further work and improvement are as follows:

- State estimation. Integrating a Kalman filter to track the source bearing would smooth the estimate and predict through brief dropouts, improving on the current memoryless smoothing.
- Stronger actuators. More powerful motors would allow faster, more forceful repositioning, reducing reaction time and improving tracking of quickly moving sources.
- Automated parameter tuning. The detection and control parameters were set by hand; an automated parametric sweep — for instance an AI-driven optimization over the threshold, persistence, smoothing, and PID gains, etc. — could find better-performing configurations than manual tuning.

- Additional estimation techniques. Other methods for improving the TDOA estimates, such as statistical outlier filtering or Lawson norm optimization [5] may be worth exploring.

7 Project Documentation

All project deliverables are documented in the project's GitHub repository; this report describes them only briefly.

7.1 Description of the project documentation

The repository contains the complete set of project deliverables: the Python perception and detection code, the STM32 control firmware, and the offline simulation environment. Together these cover the full system — live direction-of-arrival tracking and binary tone detection on the host, motor control on the microcontroller, and the simulation used to validate the algorithm.

7.2 Documentation location

The project is hosted on GitHub at:

`https://github.com/RT2718/DDaT_final_project`

7.3 Description of the project files

The repository contains three main items:

- `Final_code_Detection_Tracking.py` — the main Python program. It handles audio acquisition, binary tone detection, GCC–PHAT direction-of-arrival estimation, and streaming of the resulting bearing to the microcontroller.
- `main.c` — C code for motor control.
- `tracking_and_simulation_final.py` — the same code logic as `Final_code`, with the optional simulation mode.

References

[1] Knapp, C., and Carter, G., "The generalized correlation method for estimation of time delay", IEEE Transactions on Acoustics, Speech, and Signal Processing 24.4 (1976): 320-327.

[2] Oppenheim, A. V., and Schafer, R. W., "Discrete-Time Signal Processing", Pearson, 2009.

[3] STM32 HAL Driver Reference Manual, STMicroelectronics.

[4] x-nucleo-ihm01a1 Quick Start Guide, May 16, 2016.

[5] Greco, D., ”Robust Blind Algorithm for DOA Estimation Using TDOA Consensus”, Acoustics 2025, 7, 52.

[6] Krishnaraj Varma, Takeshi Ikuma, and A. A. (Louis) Beex, ”Robust TDE-Based DOA Estimation For Compact AUDIO Arrays”, Second IEEE Sensor Array and Multichannel Signal Processing Workshop (SAM2002), 4-6 August, 2002.

[7] J. O. Smith, ”Quadratic Interpolation of Spectral Peaks”, Spectral Audio Signal Processing, https://www.dsprelated.com/freebooks/sasp/Quadratic_Interpolation_Spectral_Peaks.html

[8] G. Hunter – ”Exponential Moving Average (EMA) Filters”, https://blog.mbedded.ninja/programming/signal-processing/digital-filters/exponential-moving-average-ema-filter/